

Demonstration

Safe Semantic Planner in a Finite Cartesian State Space

PCCST503 - Machine Learning | Assignment 1

Department of Computer Science and Engineering

1. Demonstration Overview

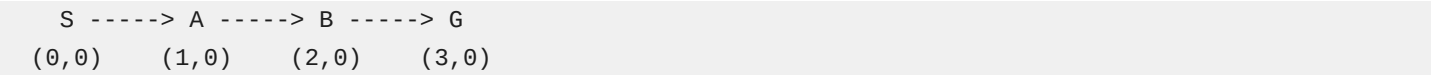
This document provides a walkthrough of the Safe Semantic Planner in action. It demonstrates the planner's core capabilities through 6 test cases, showing how the algorithm handles basic pathfinding, safety constraints, and dynamic environment changes.

2. Demo 1: Finding a Simple Path

Setup

A simple linear graph with 4 states: S(0) -> A(1) -> B(2) -> G(3). All transitions have cost 1.0 and are available. No bad states.

Graph Layout



Execution

```
$ python3 main.py

TEST CASE 1: Basic Reachability
Success:      True
State Path:   [0, 1, 2, 3]
Total Cost:   3.0
Explored States: 4
```

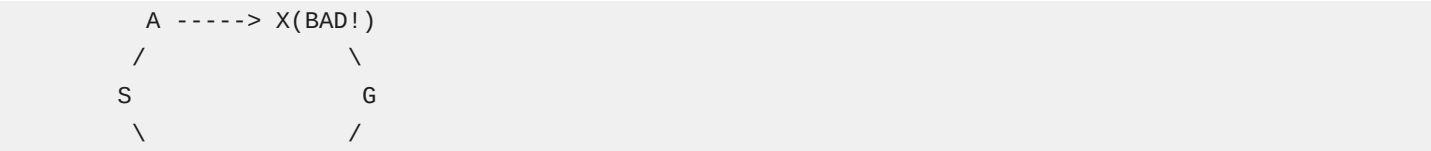
The planner correctly finds the only available path with a total cost of 3.0.

3. Demo 2: Avoiding Dangerous States

Setup

Two paths exist. Path 1 goes through bad state X, Path 2 is a safe detour. The planner must automatically avoid X.

Graph Layout



C -----> D

Result

```

State Path:      [0, 3, 5, 4]
(S -> C -> D -> G)
Bad States Visited: 0
Safety Score:     1.4142

```

The planner avoids X entirely and chooses the safe detour through C and D.

4. Demo 3: Cost vs Safety Tradeoff

Setup

Two valid paths exist (neither goes through a bad state). Path 1 is cheap but passes close to a bad state. Path 2 is expensive but stays far away.

Comparison

	Path 1 (Cheap)	Path 2 (Safe)
Route	S -> A -> B -> G	S -> C -> D -> G
Cost	3.0	6.0
Safety Distance	0.5	~3.16
Chosen?	Yes (by A*)	No

A* picks Path 1 because it has lower cost. The safety score of 0.5 warns us that this path comes very close to a bad state. By adjusting the gamma weight in the scoring function, we could make the planner prefer safer paths.

5. Demo 4: Handling Broken Connections

Scenario

The planner initially finds the direct path S -> A -> G. Then, the transition from A to G becomes unavailable (simulating a broken connection).

Before

```

Path: [0, 1, 4] (S -> A -> G)
Cost: 2.0

```

After (A -> G breaks)

```
Path: [0, 2, 3, 4] (S -> B -> C -> G)
Cost: 4.5
Replanning Time: ~0.00007 seconds
```

The planner successfully finds an alternative route after the direct connection breaks. Replanning is nearly instantaneous.

6. Demo 5: Goal Changes Mid-Execution

Scenario

The original goal is G1 (state 2). After the initial plan is computed, the goal changes to G2 (state 4). The planner replans to reach the new goal.

Before (Goal = G1)

```
Path: [0, 1, 2] (S -> A -> G1)
Cost: 2.0
```

After (Goal = G2)

```
Path: [0, 1, 3, 4] (S -> A -> B -> G2)
Cost: 3.0
Replanning Time: ~0.000025 seconds
```

The planner quickly adapts to the new goal without any issues.

7. Demo 6: Discovering New Shortcuts

Scenario

Initially, only the long path S -> A -> B -> G exists (cost=6.0). A new direct transition S -> G (cost=1.0) is then added to the graph.

Before (Long Path Only)

```
Path: [0, 1, 2, 3] (S -> A -> B -> G)
Cost: 6.0
Explored States: 4
```

After (Shortcut Added)

```
Path: [0, 3] (S -> G directly!)
```

Cost: 1.0

Explored States: 2

The planner discovers and uses the new shortcut, reducing cost from 6.0 to 1.0 and explored states from 4 to just 2.

8. Conclusion

The demonstration shows that the Safe Semantic Planner successfully handles all required scenarios:

- Finds optimal paths in simple graphs
- Avoids all bad states with zero violations
- Computes meaningful safety distance metrics
- Handles dynamic changes: broken transitions, goal updates, new shortcuts
- Replans in microseconds for small graphs
- Produces a combined score balancing cost, safety, and reliability